06 — hazard_unit.v Explained: Forwarding and Stall Decisions

Oct 6, 2026 · @shiv

1. Overview

`hazard_unit.v` watches the pipeline and makes two decisions every cycle. It chooses where each operand of the EX instruction should come from (**forwarding**), and whether the pipeline must wait one cycle after a load (**stall**). It is pure combinational logic made of comparators.

Item	Value
File	design_sources/hazard_unit.v (109 lines)
Module name	hazard_unit
Pipeline stage	Looks at ID, EX, MEM and WB at the same time
Type of logic	Combinational (register-number comparators)
Calls (instantiates)	Nothing – leaf module
Called by	risc8_5stage_core.v, instance u_hazard, lines 166–192
Gets uses_a / uses_b from	control_unit.v (via the core and the ID/EX register)

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v
    ├── control_unit.v
    ├── regfile.v
    ├── hazard_unit.v    <== THIS FILE
    ├── alu.v
    └── dmem.v
```

1.2 Ports

Port	Dir.	Width	Meaning	Connected to (in risc8_5stage_core.v)
id_valid	in	1	Real instruction in ID	ifid_valid
id_ra_addr, id_rb_addr	in	3 each	Registers the ID instruction reads	id_ra_addr, id_rb_addr
id_uses_a, id_uses_b	in	1 each	Does it really read them?	id_uses_a, id_uses_b (from control_unit)
ex_valid	in	1	Real instruction in EX	idex_valid
ex_mem_to_reg	in	1	EX instruction is a load	idex_mem_to_reg
ex_rd	in	3	Register the EX instruction writes	idex_rd
ex_ra_addr, ex_rb_addr	in	3 each	Registers the EX instruction reads	idex_ra_addr, idex_rb_addr
ex_uses_a, ex_uses_b	in	1 each	Does it really read them?	idex_uses_a, idex_uses_b
mem_reg_we	in	1	MEM instruction will write a register	exmem_reg_we
mem_mem_to_reg	in	1	MEM instruction is a load	exmem_mem_to_reg
mem_rd	in	3	Register the MEM instruction writes	exmem_rd
wb_reg_we	in	1	WB instruction writes a register	memwb_reg_we
wb_rd	in	3	Register the WB instruction writes	memwb_rd
fwd_a_sel, fwd_b_sel	out	2 each	Operand source: 00 normal, 10 EX/MEM, 01 MEM/WB	forwarding multiplexers ex_opA, ex_opB
stall	out	1	1 = hold PC and IF/ID, insert a bubble	PC / IF/ID / ID/EX update logic

1.3 Dependencies

Modules, functions or tasks called inside this file: none. It is a leaf module.

Used by (file)	How	Instance	Lines
risc8_5stage_core.v	Module instance	u_hazard	166–192
tb_risc8_5stage.v	Reads stall, fwd_a_sel and fwd_b_sel through the core, to count events	dut.stall, dut.fwd_a_sel	74–77

1.4 Verilog constructs used

Construct	Meaning in plain words
`always @(*)` with `if / else if / else`	A priority multiplexer: the first true condition wins
`==`	Equality comparator (3-bit register numbers)
`&&`, `||`, `!`	Logical AND, OR, NOT
`2'b10`	A 2-bit binary constant
`assign`	Continuous combinational output

2. Code walkthrough, part by part

2.1 File header (lines 1–40)

The header describes the two jobs: **forwarding** (no time lost) and the **load-use stall** (1 cycle lost). It lists the forwarding select codes and explains why EX/MEM has priority over MEM/WB: it holds the newer value. It notes that hazards three instructions apart are handled by the bypass in regfile.v, and that branch (control) hazards are handled in risc8_5stage_core.v, not here.

2.2 Ports (lines 41–73)

```
module hazard_unit (
    // ---- Instruction in ID (Stage 2) : for load-use detection ----
    input  wire       id_valid,      // IF/ID holds a real instruction
    input  wire [2:0] id_ra_addr,    // register read on port A
    input  wire [2:0] id_rb_addr,    // register read on port B
    input  wire       id_uses_a,     // port A really used?
    input  wire       id_uses_b,     // port B really used?

    // ---- Instruction in EX (Stage 3) ----
    input  wire       ex_valid,      // ID/EX holds a real instruction
    input  wire       ex_mem_to_reg, // EX instruction is a load (LD)
    input  wire [2:0] ex_rd,         // its destination register
    input  wire [2:0] ex_ra_addr,    // registers it reads (for forwarding)
    input  wire [2:0] ex_rb_addr,
    input  wire       ex_uses_a,     // EX instruction really reads port A?
    input  wire       ex_uses_b,     // EX instruction really reads port B?

    // ---- Instruction in MEM (Stage 4) ----
    input  wire       mem_reg_we,    // will write a register
    input  wire       mem_mem_to_reg,// is a load (its data not ready in
EX/MEM)
    input  wire [2:0] mem_rd,

    // ---- Instruction in WB (Stage 5) ----
    input  wire       wb_reg_we,
    input  wire [2:0] wb_rd,

    // ---- Outputs ----
    output reg  [1:0] fwd_a_sel,     // operand A source in EX
    output reg  [1:0] fwd_b_sel,     // operand B source in EX
    output wire       stall          // 1 = load-use stall this cycle
);
```

The inputs are grouped by pipeline stage: what the ID, EX, MEM and WB instructions read or write. The unit compares register numbers between those stages.

2.3 Forwarding select for operand A (lines 74–87)

```
    // 1) FORWARDING SELECT for operand A
    //    Forward only if the EX instruction really reads port A
    //    (ex_uses_a), so unused register fields never trigger it.
    always @(*) begin
        if (ex_uses_a && mem_reg_we && !mem_mem_to_reg && (mem_rd ==
ex_ra_addr))
            fwd_a_sel = 2'b10;          // newest value: from EX/MEM
        else if (ex_uses_a && wb_reg_we && (wb_rd == ex_ra_addr))
            fwd_a_sel = 2'b01;          // older value: from MEM/WB
        else
            fwd_a_sel = 2'b00;          // no hazard: use register value
    end
```

Read it as three questions, asked in order:

1. Does the EX instruction read port A, **and** will the instruction in MEM write a register, **and** is that instruction not a load, **and** is its register the one we read? Then select **10**: take the value from EX/MEM.
2. Otherwise, will the instruction in WB write the register we read? Then select **01**: take the value from MEM/WB.
3. Otherwise select **00**: use the value read in ID.

The order matters. If both MEM and WB write the same register, the MEM one is newer and must win. A load in MEM is skipped because its data is not ready in EX/MEM yet; the stall below makes sure no one needs it then.

Link: in risc8_5stage_core.v (lines 202–205), `fwd_a_sel` drives the multiplexer that produces `ex_opA`, the ALU's A input.

2.4 Forwarding select for operand B (lines 88–99)

```
    always @(*) begin
        if (ex_uses_b && mem_reg_we && !mem_mem_to_reg && (mem_rd ==
ex_rb_addr))
            fwd_b_sel = 2'b10;
        else if (ex_uses_b && wb_reg_we && (wb_rd == ex_rb_addr))
            fwd_b_sel = 2'b01;
        else
            fwd_b_sel = 2'b00;
    end
```

This is exactly the same logic for port B. Port B carries rs2, or the store data for ST.

2.5 Load-use stall (lines 100–109)

```
    // 2) LOAD-USE STALL
    //    LD is in EX AND the instruction in ID reads LD's destination.
    assign stall = id_valid && ex_valid && ex_mem_to_reg &&
                   ((id_uses_a && (id_ra_addr == ex_rd)) ||
                    (id_uses_b && (id_rb_addr == ex_rd)));

endmodule
```

`stall` is 1 when **all** of these hold:

- there is a real instruction in ID (`id_valid`), and a real one in EX (`ex_valid`);
- the EX instruction is a load (`ex_mem_to_reg`);
- the ID instruction really reads the register the load will write, on port A or port B.

Link: in risc8_5stage_core.v (lines 290 and 299), `stall` freezes the PC and IF/ID for one cycle and puts a bubble into ID/EX.

3. Worked examples from the test program

3.1 Forwarding from EX/MEM and MEM/WB

```
0x00  LDI R1, 0x3C
0x01  LDI R2, 0x0F
0x02  AND R3, R1, R2
```

When AND is in EX, LDI R2 is in MEM and LDI R1 is in WB. Neither has written the register file yet.

Operand	Register	Check that matches	Select	Value taken
A	R1	WB writes R1	01	MEM/WB result 0x3C
B	R2	MEM writes R2 (not a load)	10	EX/MEM ALU result 0x0F

Result: R3 = 0x3C AND 0x0F = 0x0C, with no time lost.

3.2 Load-use stall

```
0x12  LD   R5, [R4]
0x13  ADDI R5, 0xF0
```

When LD is in EX, ADDI is in ID and reads R5 on port A (ADDI uses ra_is_rd). The load writes R5, so `stall = 1`:

Cycle	ID	EX	MEM	WB
1	ADDI R5 (stall detected)	LD R5	ST	LDI R4
2	ADDI R5 (held)	bubble	LD R5 (reads memory)	ST
3	ADDI R5, 1	ADDI R5, 0xF0 (R5 forwarded, select 01)	bubble	LD R5

One cycle is lost, then MEM/WB forwarding supplies R5 = 0x0F, and ADDI gives 0xFF.

3.3 Summary of every case

Distance between writer and reader	Where the value is when the reader is in EX	Who fixes it	Cost
1 (ALU result)	EX/MEM	hazard_unit, select 10	0 cycles
1 (load)	Not ready yet	hazard_unit stall, then select 01	1 cycle
2	MEM/WB	hazard_unit, select 01	0 cycles
3	Being written in WB while the reader is in ID	regfile.v bypass	0 cycles
4 or more	Already in the register file	Nothing needed	0 cycles

4. Common mistakes

- Checking WB before MEM: when both write the same register, the older value would win, which is wrong.
- Forwarding from EX/MEM when the instruction there is a load: its data is not in EX/MEM yet.
- Not checking `valid` or `uses_a` / `uses_b`: bubbles or unused register fields cause false forwards and needless stalls.
- Forgetting that register R0 is an ordinary register here. In RISC-V, R0 is hard-wired to zero and must never be forwarded; this design does not have that rule.

5. Viva questions

Q1. What is a data hazard? An instruction needs a register value that an earlier instruction has computed but not yet written back.

Q2. What do the codes 10, 01 and 00 mean? 10 takes the value from EX/MEM, 01 from MEM/WB, and 00 uses the value read from the register file.

Q3. Why does EX/MEM have priority? It holds the most recent result. If two older instructions write the same register, the newer one must win.

Q4. Why can't forwarding fix a load followed by its use? The load's data appears only at the end of MEM, but the next instruction needs it at the start of EX in the same cycle. One cycle of waiting is unavoidable.

Q5. What does the stall do in hardware? It holds the PC and IF/ID for one cycle and inserts a bubble into ID/EX.